

A Development Workplan for a Python-Based Traveller Planetary System Generator

This document provides a comprehensive workplan for the development of a Python-based application, intended for use within a Jupyter Notebook environment, to procedurally generate detailed planetary systems according to the rules set forth in the *Traveller World Builder's Handbook*.¹ The plan is structured in a phased approach, beginning with foundational software architecture and proceeding through the sequential generation of stars, system architecture, world placement, and final detailing. The ultimate goal is a tool that is not only faithful to the source material but also robust, extensible, and useful for the end-user's creative world-building process.

I. Foundational Architecture: Data Structures and Core Utilities

Objective: To establish a robust and extensible software foundation by defining the core data models and essential utility functions. This initial phase is critical for ensuring that the complex, hierarchical data generated by the system is managed in a clean, object-oriented manner.

1.1. System Data Model Definition

The procedural generation process detailed in the handbook results in a complex hierarchy of celestial bodies, each with numerous physical and orbital attributes. To manage this complexity effectively, an object-oriented data model is essential. A series of Python classes will be defined to represent each distinct entity within a stellar system. This approach ensures data integrity, encapsulates related properties and methods, and simplifies the implementation of the generation logic in subsequent phases. The class structure must be

capable of representing intricate relationships, such as planets orbiting composite stellar groups (e.g., Aab or AB as seen in the Zed example system).¹

The data models must also be designed with future use in mind. The handbook explicitly notes a design goal of maintaining compatibility with existing Traveller community resources, such as Travellermap.com and the Traveller 5 (T5) data formats.¹ A simple script that only outputs the final game codes would be of limited utility. A truly expert implementation anticipates that the user will want to export and use this data elsewhere. Therefore, the data classes must be designed to store not only the derived game codes (e.g., Size 'A') but also the underlying physical data (e.g., diameter in kilometers, mass in Terran units) from which those codes are derived.¹ This dual storage approach will allow for the creation of flexible output modules capable of formatting the data for various external tools, significantly increasing the project's value.

Table 1: Core Data Model Specification

Class	Attribute	Data Type	Description & Source Reference
StellarSystem	name	str	The name of the system.
	stars	list	A list of all stars in the system.
	worlds	list	A flat list of all worlds for easy iteration.
	total_worlds	int	Total count of planets, giants, and belts. ¹
	baseline_number	int	System baseline number (N). ¹
	spread	float	System orbital spread (S). ¹
Star	designation	str	Stellar designation

			(A, B, Aa, Cab, etc.). ¹
	spectral_type	str	e.g., G2V, M5III. ¹
	mass	float	Mass in solar masses. ¹
	luminosity	float	Luminosity in solar units. ¹
	orbit_num	float	Orbit# around its primary, if applicable. ¹
	eccentricity	float	Orbital eccentricity, if applicable. ¹
	orbiting_bodies	list	Worlds orbiting this star or star group.
PlanetaryBody	designation	str	e.g., Aab IV, B I, Cab PI. ¹
(Base Class)	parent_star	Star	The star or star group it orbits.
	orbit_num	float	The body's Orbit#. ¹
	orbit_au	float	Orbital distance in AU. ¹
	eccentricity	float	Orbital eccentricity. ¹
	period_years	float	Orbital period in standard years. ¹
	size_code	str	Traveller size code (e.g., 8, A, S, GS4). ¹

	diameter_km	float	Diameter in kilometers. ¹
	mass_terr	float	Mass in Terran units. ¹
	satellites	list	A list of moons and/or rings.
Satellite	designation	str	e.g., 'a', 'b' for moons. ¹
	size_code	str	'S', 'R', or numerical size. ¹
	is_ring	bool	True if the satellite is a ring system.

1.2. Core Utility Library

A significant portion of the handbook's procedures involves repeated dice rolls, lookups on standardized tables, and specific mathematical conversions. To adhere to sound software engineering principles and avoid code duplication, a centralized utility library (e.g., `utils.py`) will be created. This module will house all common, low-level functions used throughout the generation process.

Key functions to be included in this library are:

- **Dice Rolling:** A flexible function or class to handle all dice notations specified in the text (e.g., 1D, 2D, D3, d10) and to correctly apply positive or negative dice modifiers (DMs).
- **Orbit# to AU Conversion:** A function implementing the precise rules for converting the game-specific Orbit# value into the standard scientific unit of Astronomical Units (AU). This is a critical prerequisite for any physics-based calculations and must follow the fractional conversion logic described on page 26.¹
- **AU to Orbit# Conversion:** The inverse of the above function, necessary for steps such as determining the Minimum Allowable Orbit (MAO) from a star's diameter given in AU. The logic for this is detailed on page 27.¹
- **Habitable Zone Calculation:** A function that takes a star's (or star group's) total luminosity and calculates the Habitable Zone Center Orbit (HZCO). This Orbit# serves as a crucial anchor point for determining the overall temperature profile of the system's

planets.¹

II. Phase 1: Stellar Body Generation

Objective: To implement the procedures for creating the star or stars that form the gravitational center of the system. This phase corresponds to the "STARS" chapter of the handbook.¹

2.1. Primary Star Generation

This initial module will implement the sequence of table lookups and dice rolls required to generate the system's primary star. The process involves determining the star's spectral type (O, B, A, F, G, K, M), its luminosity class or size (e.g., Ia, III, V, D for white dwarf), and any peculiar characteristics.¹ Following the generation of these codes, the corresponding physical properties—mass, diameter, and luminosity, typically in solar units—will be calculated or looked up from the provided tables.

2.2. Secondary and Companion Star Generation

Building upon the primary star, this module will implement the logic for determining the presence, quantity, and type of any additional stars in the system. This will generate binary, trinary, or even more complex multi-star systems. For each additional star, its orbital relationship to its own primary must be determined (Companion, Close, Near, or Far), along with its specific orbital parameters, including period and eccentricity, as per the rules on page 13.¹

2.3. Multi-Star System Logic and Designation

In systems with more than one star, the gravitational landscape is more complex. For the

purposes of planetary orbit calculation, certain groups of stars must be treated as a single composite object. For instance, a primary star Aa and its close companion Ab form a composite gravitational point Aab whose combined mass and luminosity dictate the orbits of circumbinary planets.¹ This module must contain logic to identify these groupings and calculate their composite properties.

Furthermore, a dedicated function is required to handle the complex stellar designation scheme outlined on page 13.¹ This function will correctly assign labels such as A, B, Aa, Ab, Aab, AB, and Cab. This is a crucial step for correctly identifying parent bodies when planets are placed later in the process.

The handbook presents two distinct methods for determining which orbits are stable and available for planets in a multi-star system. The first is a simplified, game-oriented method based on excluding integer Orbit# ranges.¹ The second is described as a "more realistic physics-based approach" that uses Hill sphere calculations in AU.¹ This provision of multiple complexity levels is a core design philosophy of the handbook, intended to cater to different user needs.¹ A basic implementation might choose one method, but a truly expert-level tool should honor this philosophy by implementing both. The main system generation function should therefore accept a parameter (e.g., `stability_model='simple'` or `stability_model='physics'`) to allow the user to select the desired level of realism for their system. This makes the final tool far more versatile, serving both the game master who needs a quick result and the dedicated world-builder who desires greater physical fidelity.

III. Phase 2: System Architecture and Population

Objective: To determine the number and type of worlds in the system and to calculate the specific orbital slots where they can stably exist. This phase is algorithmically intensive, translating the procedures on pages 36-50 into functional code.¹

3.1. World Count Determination

The first step is to establish the system's population. This involves a series of dice rolls to determine the quantity of the three main types of worlds:

1. **Gas Giants:** A roll determines if gas giants are present, and a subsequent roll on the "Gas Giant Quantity" table determines how many. This roll is subject to various Dice

Modifiers (DMs) based on stellar type and the number of stars in the system.¹

2. **Planetoid Belts:** A similar two-step process determines the number of planetoid belts, again with DMs applied.¹
3. **Terrestrial Planets:** The number of terrestrial planets is determined by a 2D-2 roll, with DMs for factors such as the presence of post-stellar objects.¹

The sum of these three quantities gives the Total Worlds count, a fundamental variable for the subsequent steps of planetary placement.

3.2. Orbital Slot Calculation: Available Orbits

This module is one of the most critical and complex components of the generator. It must calculate the valid Orbit# ranges where planets can exist. The logic differs significantly between single and multi-star systems.

- **Single Star Systems:** The process is straightforward. The only restriction is the Minimum Allowable Orbit (MAO), which is derived from the star's Roche limit to prevent tidal shredding of close-in worlds.¹
- **Multi-Star Systems:** This case requires a more sophisticated algorithm. The code must implement the rules for "forbidden zones" created by the gravitational influence of secondary stars. For instance, a secondary star at Orbit 6.10 with low eccentricity blocks planetary formation in the range from Orbit 5.10 to 7.10.¹ The size of this forbidden zone increases with the star's orbital eccentricity. The function for this module must process the locations and eccentricities of all secondary stars and return a list of valid, non-overlapping Orbit# ranges (e.g., [(0.61, 5.10), (7.10, 10.10)]).

3.3. Baseline and Spread Implementation

This module implements the core logic that governs the distribution and spacing of planets within the available orbits. The process begins by determining a baseline number via a 2D roll.¹ This single value dictates whether the system will be predominantly hot, cold, or temperate. The implementation must follow one of three conditional paths based on this number¹:

1. **Temperate System (baseline number is between 1 and Total Worlds):** The Baseline Orbit# (the orbital location of a specific planet in the sequence) is anchored near the star's Habitable Zone Center Orbit (HZCO).
2. **Cold System (baseline number is less than 1):** All worlds are placed in orbits farther

out than the HZCO.

3. **Hot System (baseline number is greater than Total Worlds):** All worlds are placed in orbits closer than the HZCO.

Once the Baseline Orbit# is established, the system spread is calculated. This value, representing the average separation between orbits, is derived from a formula involving the Baseline Orbit#, the MAO, and the baseline number.¹ Finally, the code will iterate from the MAO outwards, repeatedly adding the spread (plus a small random variance for realism) to generate the final list of all potential orbital slots for the system.

3.4. Handling Orbital Anomalies and Empty Orbits

Before worlds are placed, the generated list of orbital slots must be modified to account for potential irregularities, which add variety to the system architecture.

- **Empty Orbits:** The procedure calls for a roll to determine the number of empty orbits. Each empty orbit adds an additional "orbital slot" to the system's total, effectively increasing the spacing between the actual worlds.¹
- **Anomalous Orbits:** A roll is made to check for anomalous orbits. If one is generated, a subsequent roll on the "Anomalous Orbit Type" table determines its specific nature (e.g., random, eccentric, inclined, retrograde, or trojan).¹ This event adds both a new terrestrial planet to the world count and a new, randomly placed orbital slot to the system. The logic must handle the unique consequences of each type, such as applying a large DM to the eccentricity roll for an "eccentric" orbit or adding an 'R' postscript to the orbit number for a "retrograde" orbit.

IV. Phase 3: World Placement and Orbital Finalization

Objective: To populate the generated orbital architecture with the worlds counted in Phase 2, and to finalize their specific orbital characteristics.

4.1. The Placement Algorithm

This module implements the procedure described in "STEP 8: PLACING WORLDS".¹ A master list of all determined orbital slots (including those added for empty and anomalous orbits) is compiled. The random selection method described in the handbook, which uses a dice roll for a prefix and a second for the slot within that group (e.g., "3:5" in the Zed example), will be implemented to choose a slot.¹

Worlds are then placed into these randomly selected slots in a specific, fixed order:

1. Mainworld (if using the Continuation Method for existing systems)
2. Empty Orbits
3. Gas Giants
4. Planetoid Belts
5. Terrestrial Planets (filling all remaining slots)

The algorithm must handle "collisions": if a randomly selected slot is already occupied, the placement automatically moves to the next available slot in the sequence.

A subtle but critical rule on page 51 dictates that if a gas giant is randomly placed in the same orbit as a pre-existing mainworld, the mainworld becomes a moon of that giant, and "the system gains one more terrestrial planet to place in an orbit".¹ This creates a feedback loop where the total world count can change *during* the placement process. A naive implementation that assumes a static world count would fail. The workplan must therefore specify that the list of "worlds to be placed" is mutable. The placement function must be capable of adding a new terrestrial planet to its queue if this specific condition is met, ensuring the final system population is correct.

4.2. Finalizing Orbital Parameters

Once all worlds are placed, their final orbital parameters are determined.

- **Eccentricity:** For each placed world (excluding planetoid belts), a roll is made on the "Eccentricity Values" table to determine its orbital eccentricity. Any DMs resulting from the generation of anomalous orbits must be applied here.¹
- **Orbital Period:** A function will be implemented to calculate the orbital period (the length of a planet's "year"). This function must be able to handle three distinct physical scenarios as described on page 53¹:
 1. A planet orbiting a single star, using the formula .
 2. A planet orbiting a group of stars, where the total mass of all interior stars is summed: .
 3. A massive planet orbiting a star, where the planet's own mass is not negligible and must be included in the calculation.

V. Phase 4: Planetary Body Detailing

Objective: To generate the first layer of physical details for each world placed in orbit, including its size and any satellites it may possess.

5.1. World Sizing

This module will implement the sizing procedures from the "Basic World Sizing" section of the handbook.¹

- **Terrestrial Worlds:** For each terrestrial planet, a roll determines its Size code, which can range from O (a small, airless rock) to F (a massive "mega-earth").
- **Gas Giants:** For each gas giant, a roll first determines its general category: Small (GS, Neptune-like), Medium (GM, Jupiter-like), or Large (GL, Super-Jovian). Subsequent rolls are then made to determine its specific diameter and mass within the ranges for that category.¹ The detailed rolls for the Zed system on page 57 provide excellent test cases for verifying this logic.¹

5.2. Satellite System Generation

This module will implement the rules for generating moons and ring systems for each planet and gas giant.¹ The process is two-fold:

1. **Quantity:** The number of significant satellites is determined by a roll on a table, with the result depending on the parent planet's size.
2. **Size/Type:** For each potential satellite, a further roll determines its size. The result can be a numerical size (e.g., 1-A), 'S' for a small, captured body, or 'R' for a ring system.

The handbook also provides a simple rule of thumb for determining the potential number of *insignificant* moons (small, often captured asteroids), which can be implemented as a simple calculation based on the parent body's size.¹

The purpose of the handbook is not merely to generate sterile data but to assist a Referee in

creating an interesting setting for adventure.¹ The author's own commentary often points the user toward narratively interesting results. For instance, after generating moons for the Zed system, the text explicitly notes that "By chance all three of these large(ish) moons are theoretically within the habitable zone... These three moons... are among the best candidates for Mainworld".¹ An expert-level tool should automate this kind of guidance. Therefore, the workplan includes a post-generation analysis function. This function will iterate through all generated bodies and apply a set of heuristics to flag "Points of Interest." It would check for conditions like large moons in the habitable zone, planets with an unusually high number of moons, or worlds in highly eccentric or retrograde orbits, and add a descriptive flag to these bodies in the final output. This transforms the tool from a simple data generator into a creative assistant.

VI. Phase 5: System Output and Presentation

Objective: To format the complete, generated system data into clear, readable, and useful formats as specified in the handbook.

6.1. Data Aggregation and Designation

As a final processing step, a function will iterate through the completed object model of the system to assign the standard Traveller designations to all generated bodies. This involves processing the orbital slots for each star (or star group) in order, assigning Roman numerals to planets and P + Roman numeral to planetoid belts.¹ Moons are assigned sequential alphabetic characters based on their distance from the primary.¹

6.2. Generating System Profiles

The application will include functions to generate the two summary profiles described on page 58¹:

- **Short Profile:** A concise string summarizing the system's composition in the format G-P-T-N-S (Gas Giants-Planetoid Belts-Terrestrial Planets-Baseline Number-Spread).
- **Long Profile:** A more detailed string that shows the sequence of world types in their

orbits around each star, such as Aab-5-T-T-T-P-G-G-T-T-T-T-G-0.5.

6.3. Tabular Data Presentation

The primary output within the Jupyter Notebook environment will be a comprehensive, human-readable table. The use of the Pandas library is recommended for this purpose, as it provides excellent display capabilities and allows for easy export to common data formats like CSV. The table structure will mirror the "SOL SYSTEM OVERVIEW" example on page 68, providing a familiar and intuitive layout for users.¹ Key columns will include: Primary (the parent star or star group), Object (the world's designation), Orbit#, AU, Eccentricity, Period, SAH/UWP (Size code), Sub (number of significant moons), and a Notes field to display any flagged "Points of Interest."

Table 2: Procedural Implementation Map

This table serves as the central, actionable checklist for the development process, mapping each step from the handbook to a specific software component.

Step ID	Process Step Description	Proposed Function Name	Inputs	Outputs	Source Reference (Page/Table)
PHASE 1	(Stellar Body Generation)				
1.1	Generate Primary Star Type/Size	generate_primary_star()	None	Star object	¹ , p. 13
1.2	Determine Number of Companion	determine_companion_count()	Star object	int	¹ , p. 13

	Stars				
1.3	Generate Secondary Star Properties	generate_secondary_star()	Star object (primary)	Star object (secondary)	¹ , p. 13
1.4	Calculate Composite Star Properties	calculate_composite_stars()	list	list (with composites)	¹ , p. 13, 38
PHASE 2	(System Architecture)				
2.1	Determine Gas Giant Quantity	calculate_gas_giant_count()	StellarSystem object	int	¹ , p. 38
2.2	Determine Planetoid Belt Quantity	calculate_belt_count()	StellarSystem object	int	¹ , p. 38
2.3	Determine Terrestrial Planet Quantity	calculate_terrestrial_count()	StellarSystem object	int	¹ , p. 38
2.4	Calculate Available Orbits (Simple)	get_available_orbits_simple()	StellarSystem object	list[tuple(min, max)]	¹ , p. 38-39
2.5	Calculate Available Orbits (Physics)	get_available_orbits_physics()	StellarSystem object	list[tuple(min, max)]	¹ , p. 40

2.6	Determine Baseline Number & Orbit	calculate_baseline()	StellarSystem object	float (Baseline Orbit#)	¹ , p. 43-45
2.7	Calculate System Spread	calculate_spread()	StellarSystem object	float (Spread)	¹ , p. 48
2.8	Generate All Orbital Slots	generate_orbital_slots()	StellarSystem object	list[float]	¹ , p. 48
2.9	Handle Empty & Anomalous Orbits	modify_slots_for_anomalies()	StellarSystem, list[float]	list[float], list[AnomalousOrbit]	¹ , p. 48-50
PHASE 3	(World Placement)				
3.1	Place Worlds into Orbital Slots	place_worlds()	StellarSystem object	dict (slot -> world_type)	¹ , p. 51-53
3.2	Determine Orbital Eccentricity	determine_eccentricity()	PlanetaryBody object	float	¹ , p. 52
3.3	Calculate Orbital Period	calculate_period()	PlanetaryBody object	float	¹ , p. 53
PHASE 4	(Planetary Detailing)				
4.1	Determine	determine_	PlanetaryBody	str (size code),	¹ , p. 54-57

	World Size	world_size()	ody object	floats (phys. data)	
4.2	Generate Satellites	generate_satellites()	PlanetaryBody object	list	¹ , p. 56, 59
4.3	Flag Points of Interest	flag_points_of_interest()	StellarSystem object	Modifies Notes attribute on bodies	¹ , p. 59
PHASE 5	(System Output)				
5.1	Assign Final Designations	assign_designations()	StellarSystem object	Modifies designation attributes	¹ , p. 52, 59
5.2	Generate System Profiles	generate_profiles()	StellarSystem object	str (short), str (long)	¹ , p. 58
5.3	Create Output DataFrame	create_system_dataframe()	StellarSystem object	pandas.DataFrame	¹ , p. 68

Works cited

1. World_Builder_s_Handbook.pdf